Name: Student ID:

Agentic AI for Business and FinTech (SEEM5660)
Individual Homework 03, Due date: 16 November, 23:59 HKT.

Individual homework. You are encouraged to discuss with others; however, you must submit your own work.

1 Overview

Read the book "Common Stocks and Uncommon Profits" by Philip Fisher[1]. In Part One, Chapters 3 and 4 [What to buy], Fisher lays out a framework for analyzing asset selection: 15 points to look for in a common stock, from market potential (point 1) to management integrity (point 15).

Your task is to implement an AI agent that automates this process for **3 points of your choice**. For every (fictional) company, your agent researches the company with tools, gives a PASS or FAIL verdict for each of your 3 points, and writes a short summary. (PASS means the company is performing well for a point, and you decide your own criterion for a PASS)

Part	What you do	Points
Task 1	Build a Fisher agent for 3 points of your choice	75
Report	Describe your agent and its results	15
Task 2	Reflection	10

If you encounter any problems with this homework, inform the TAs immediately: Binh Nguyen (`tbnguyen@se.cuhk.edu.hk`, WeChat ID: `binhnguyen2702`) and Bao Nguyen (`nbnguyen@se.cuhk.edu.hk`, WeChat ID: `baonn7`).

2 Setup

Fork `https://github.com/BINH2702/FTEC5660-HW3`, then (Python 3.10 or newer):

```
git clone <your fork's url>
cd FTEC5660-HW3
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
echo "DEEPSEEK_API_KEY=your_key_here" > .env
```

The repo contains `hw3.py` (the starter code, which also builds the four tools) and `public_test/` (20 sample companies with `ground_truth.json`). Use your own DeepSeek API key (`https://platform.deepseek.com`). Never commit `.env`; it is already in `.gitignore`.

You must use `deepseek-v4-flash` (the only model allowed) and LangChain to build your agent. Company data may be read **only through the four tools** provided in `hw3.py` (Appendix A): do not open `company.db` or `interviews.dat` yourself, and do not use web search or any other data source. The companies are fictional, so the model cannot answer from memory.

[1] A digital copy could be found at `https://drive.google.com/file/d/1v2nMaz9pjnrCtMwB1kLWod4uOBJYmJiV/view?usp=share_link`

3 Task 1: Fisher Agent for 3 Points (75 points)

3.1 Problem

For each company, your agent must:

1. give a verdict, `PASS` or `FAIL`, on each of your 3 chosen points, using the rule you designed for it (Section 3.3); PASS means the company looks favourable on that point;
2. write a short summary that explains the verdicts.

The evidence is spread over three sources, and your agent has to go and get it:

- **Documents** (`list_documents`, `read_document`): a 2025 letter to shareholders, four earnings calls (Q3 2025 to Q2 2026) and a news file (July 2025 to July 2026). These are management's own words, so they are not evidence on their own: check what management claims against the database and the interviews.
- **Database** (`sql_query`): a read-only SQLite database with the company's orders, monthly costs, cash flow, capex plan, product returns, insider share trades and the industry medians. At most 50 rows come back per query, so use `SUM`, `GROUP BY` and `WHERE`.
- **Interviews** (`interview(role, topic)`): what a customer, a current or former employee, a competitor or a supplier says about one topic, at most **10 interviews per company**. Only some people know about each topic (Section 3.3).

Example (`public_test/hollins_group`, medical devices). Suppose you choose point 4, "Does the company have an above-average sales organization?". A customer says "Their salespeople just send the price list; we go to rivals for advice", so a sensible rule marks point 4 `FAIL`, and `ground_truth.json` agrees. Now suppose you also choose point 5, "Does the company have a worthwhile profit margin?". You have to decide what "worthwhile" means: a fixed percentage, the company's own past, or other companies in its industry? Try your choice on the 20 public companies and compare your verdicts with `ground_truth.json`.

3.2 Terms used in the data

Term	Meaning
FY2025	Fiscal year 2025. Here the fiscal year is the calendar year, so FY2025 is January to December 2025.
Quarter	Three months of the year: Q1 is January to March, Q2 April to June, Q3 July to September and Q4 October to December. The database writes `2025Q3`; the documents write "third quarter 2025".
Amounts	The database is in **USD thousands** (676,100 means US$676.1 million). The documents use USD millions.
Revenue	Money from sales: the sum of `amount` in `orders`.
Cost of sales	The cost of making the products that were sold.
Selling and admin	The cost of selling and of running the company (sales staff, offices, management).
R&D	Research and development spending, mainly on new products.
Operating margin	Operating profit as a share of revenue: (revenue − cost of sales − selling and admin − R&D) ÷ revenue.
Operating cash flow	Cash that the day-to-day business brings in during a quarter.

Term	Meaning
Cash at the end of a quarter	The cash the company holds on the last day of that quarter (`cash_end`).
Capex	Capital expenditure: spending on factories, machines and other long-term assets. `capex_plan` holds the planned capex for 2026 to 2028.
Return rate	Units returned ÷ units shipped, from the `returns` table.
Industry median	The middle value for companies in the same industry, from `industry_medians`.
Insider	A director or senior manager of the company. `insider_trades` lists the shares they bought (`buy`) or sold (`sell`).
Earnings call	A meeting each quarter where management presents the quarter's results to investors. Each call quotes that quarter's revenue.

3.3 Choose your 3 points and design your rules

Pick any 3 of Fisher's 15 points. The full questions are in the book and in `FISHER_POINTS` in `hw3.py`. For each of your points, **you design the rule**: which evidence to use, what to compute or whom to ask, and where the line between PASS and FAIL lies.

#	Fisher's point	Where the evidence is
1	Market potential for sales growth	database
2	New products as current lines mature	database
3	Effective research and development	database
4	Above-average sales organization	interviews (`sales_team`)
5	Worthwhile profit margin	database
6	Maintaining or improving margins	database
7	Labour and personnel relations	interviews (`working_conditions`)
8	Executive relations	interviews (`top_management`)
9	Depth of management	interviews (`decision_making`)
10	Cost analysis and accounting controls	earnings calls, checked against the database
11	Advantage over competitors	interviews (`competitive_edge`)
12	Long-range outlook on profits	database
13	Growth without dilutive equity financing	database
14	Management talks freely about troubles	earnings calls, checked against the database
15	Management of unquestionable integrity	news file, checked against the database

Interviews. Only some people know about each topic; anyone else says they cannot comment, and that answer still uses up one of your 10 interviews. People who know always agree, so one good answer is enough.

Topic	Who knows about it
`sales_team`	customer, competitor
`working_conditions`	current_employee, former_employee
`top_management`	former_employee, supplier
`decision_making`	supplier, customer
`competitive_edge`	competitor, customer

How your rules are graded. We grade your verdicts against our own answer key. It uses one fixed rule per point, which we do not publish. The data never puts a company close to the line of that rule, so a sensible reading of Fisher usually gives the same answer. **Test your rules on the public answers**: `public_test/ground_truth.json` gives the expected verdict on all 15 points for each of the 20 public companies, and every point fails in 6 of them. If your rule disagrees with the public answers, rethink it.

Tip. Do every sum, ratio and date difference in SQL or Python, not in the model's head (SQLite's `julianday()` turns a date into a number of days). In our tests, arithmetic done by the model was the most common cause of wrong verdicts.

3.4 Your implementation

Fill in the three places in `hw3.py` marked `### YOUR CODE HERE`. You may add imports, constants and helper functions above them, but do not change the provided code below them (the tools, the runner and the scoring):

- `CHOSEN_POINTS`: your 3 points, e.g. `CHOSEN_POINTS = [1, 7, 15]`.
- `build_agent()`: create your model, prompt(s) and anything else you need once.
- `analyze_companies(agent, companies)`: `companies` maps each company folder to its four tools, already wrapped as LangChain tools, e.g. {`"hollins_group": [list_documents, read_document, sql_query, interview]`}. Return, for every company, {`"points": {1: "PASS", 7: "FAIL", 15: "PASS"}, "summary": "..."`}. Catch errors per company (e.g. a failed API call) and still return an entry, so that one failure does not crash the whole run.

Use the techniques from the course where they fit your points: tool use for the database and the interviews, planning which interviews to spend, routing each point to the right source, prompt chaining, parallel calls, reflection to check your answers, or a combination. Do not hard-code folder names, company names or the public answers. Run your agent on the public companies with:

```
python3 hw3.py --data-folder public_test
```

This writes `results.csv`, checked automatically against `public_test/ground_truth.json`, with the columns `company`, `points`, `points_correct`, `point_errors`, `interviews_used` and `summary`, and prints your point score on your 3 points. Once `CHOSEN_POINTS` is set, the starter runs before you add any API code (it returns no verdicts, so it scores 0%).

3.5 Evaluation

We grade Task 1 by running your code on a private set of about 20 companies that you have not seen. They use the same tools and the same answer key rules as `public_test`; only the company names and the wording are different. We grade your 3 points only. We run your code three times and take the average.

Your Task 1 score is your **point score** times 75. The point score is the average of two shares: the share of truly passing verdicts that you marked PASS and the share of truly failing verdicts that you marked FAIL. For example, 90% and 60% give $(90\% + 60\%)/2 = 75\%$, which is worth $0.75 \times 75 = 56.25$ points. Marking every point PASS gives $(100\% + 0\%)/2 = 50\%$, worth only 37.5 points. (`hw3.py` prints the point score after "balanced".)

A missing or invalid answer counts as wrong. Each run must finish within 30 minutes.

Your submission must run end-to-end and produce `results.csv`.
A run that crashes, exceeds 30 minutes, or produces no `results.csv` scores **zero** on Task 1.
Test it from a fresh clone before you submit.

4 Report (15 points)

Write your report in a "Homework 3 solution" section of your `README.md`:

- for each of your 3 points: why you chose it, your PASS rule in one or two sentences, and the evidence it uses (quote Fisher where it helps);
- a visualization of your agent design (for example, which tools and model calls handle which points) and one paragraph describing it;
- your point score on `public_test`, the companies where your verdicts disagree with the public answers, and why.

5 Task 2: Reflection (10 points)

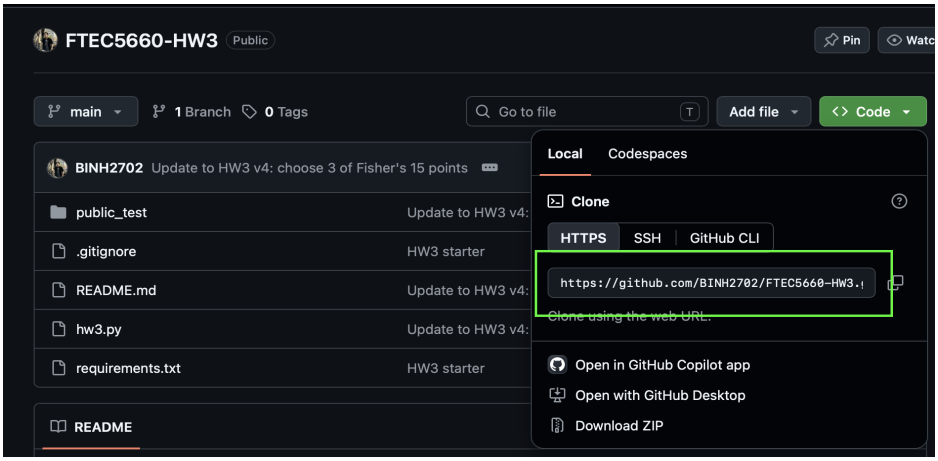
Submit your reflection on how AI-sycophancy and AI-psychosis. No restrictions on the format; you can write/draw/or submit an AI-generated submission.

6 Submission

Submit your fork, a public GitHub repo named **FTEC5660-HW3**, by entering your GitHub link as shown in the images below. We grade the last commit before the deadline. The repo contains:

- `hw3.py` with your Task 1 agent,
- `requirements.txt`, with any extra packages you use added,
- `README.md` with your report.

Do not put any API key in your code: we run your code with our own key. Keep the repo public during grading. For students unfamiliar with GitHub, we recommend this tutorial: `https://www.youtube.com/watch?v=-RZ03WHqkaY`.



A Tools

For each company folder, `hw3.py` builds four LangChain tools with `make_tools(company_folder)` and passes them to `analyze_companies` in this order: `list_documents`, `read_document`, `sql_query`, `interview`. Every tool returns a string. The examples below use `public_test/hollins_group`.

A.1 Tool 1: `list_documents`

Purpose: List the company's six documents with their titles and dates.

Input Parameters: none.
Output (example):

```
annual_letter_2025.md: Hollins Group: letter to shareholders, 2025 (Date: 2026-03-12)
earnings_call_2025q3.md: Hollins Group: earnings call, third quarter 2025 (Date: 2025-10-29)
earnings_call_2025q4.md: Hollins Group: earnings call, fourth quarter 2025 (Date: 2026-02-25)
earnings_call_2026q1.md: Hollins Group: earnings call, first quarter 2026 (Date: 2026-04-28)
earnings_call_2026q2.md: Hollins Group: earnings call, second quarter 2026 (Date: 2026-07-29)
news_2025_2026.md: Hollins Group: news, July 2025 to July 2026 (undated)
```

A.2 Tool 2: `read_document`

Purpose: Return the full text (Markdown) of one document.

Input Parameters:

```
{
    "name": str   # e.g. "annual_letter_2025.md"
}
```

Output: the document text. An unknown name returns a message that lists the available names.

A.3 Tool 3: `sql_query`

Purpose: Run one read-only SQLite query on the company's database.

Input Parameters:

```
{
    "query": str   # one SQLite SELECT statement
}
```

Output: a header line and at most 50 rows, columns separated by `|`, decimals rounded to 2 places. If there are more rows, a last line says so; aggregate, or page with `LIMIT/OFFSET`. An invalid query returns a message starting with `SQL error:`.

Example:

```
sql_query("SELECT substr(order_date,1,4) AS year,
           ROUND(SUM(amount)/1000,1) AS revenue_musd FROM orders GROUP BY year")

year | revenue_musd
2023 | 437.60
2024 | 477.90
```

```
2025 | 522.00
2026 | 290.40
```

Amounts are in USD thousands; dates are `YYYY-MM-DD`, months `YYYY-MM` and quarters like `2025Q3`; the fiscal year is the calendar year.

Table	Columns
`company`	name, industry
`industry_medians`	industry, revenue_growth_2y_pct, new_product_share_pct, new_product_sales_per_rd, operating_margin_pct (one row per industry; use the row for the company's own industry)
`orders`	order_id, order_date, product, customer, amount, note (about 3,000 to 4,000 orders from January 2023 to June 2026; revenue is the sum of order amounts)
`products`	product, launch_date
`monthly_costs`	month, cost_of_sales, selling_admin, research_development
`quarterly_cash`	quarter, operating_cash_flow, capex, cash_end
`capex_plan`	year, planned_capex (2026 to 2028)
`returns`	month, units_shipped, units_returned
`insider_trades`	trade_date, person, title, side (`buy` or `sell`), shares, price

A.4 Tool 4: `interview`

Purpose: Hear what one person says about one topic (Fisher's "scuttlebutt"), at most 10 interviews per company. Section 3.3 lists who knows about each topic.

Input Parameters:

```
{
    "role": str,    # customer, current_employee,
                    # former_employee, competitor, supplier
    "topic": str    # sales_team, working_conditions,
                    # top_management, decision_making,
                    # competitive_edge
}
```

Output: one quoted answer. An unknown role or topic returns an error message and is not counted. After 10 interviews, every call returns `Interview limit reached`.

Example:

```
interview("customer", "sales_team")
A buyer at a buying consortium: "Their salespeople just send the price list; we
go to rivals for advice."

interview("competitor", "working_conditions")     # a competitor does not know
A sales director at a rival firm: "We only see them in tenders; I can't comment
on that."
```